

```

21 stage('build') {
22     steps {
23         script {
24             image = docker.build("${IMAGE}")
25             println "Newly generated image, " + image.id
26         }
27     }
28 }

```

Figure 8: The build image

reading on this topic, refer to Jenkins documentation on using Docker with Pipeline, which provides documentation with code samples, which are very handy, and covers various use cases.

Docker build

The build image (Figure 8) shows the `docker.build()` running without any arguments other than `$(IMAGE)`; however, it can take additional arguments via `--build-arg`. This behaviour is identical to the Docker build, so it must end with the build context. The output is saved to a variable, which contains the image object that will be used in the following stages.

The console output (Figure 9) showing the execution of the build stage helps to trace the sequence of events in the build console log from a successful execution of the build stage, as per the code snippet shown in Figure 8. From the log, it's clear that Docker has used the cache while executing one of the steps, because there were a few repeated runs of the build job (on the same build agent).

Image validation

Remember the note about 'Hello World' mentioned earlier

```

[Pipeline] stage
[Pipeline] { (build)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
13:52:55 [tutum-hello-world-anyagent] Running shell script
13:52:55 + docker build -t raspamdocker/osfy .
13:52:56 Sending build context to Docker daemon 223.2kB
13:52:56 Step 1/2 : FROM tutum/hello-world
13:52:56 ----> 31e17b0746e4
13:52:56 Step 2/2 : EXPOSE 80
13:52:56 ----> Using cache
13:52:56 ----> 5a0f79a022d
13:52:56 Successfully built 5a0f79a922d
13:52:56 Successfully tagged raspamdocker/osfy:latest
[Pipeline] dockerfingerprintfrom
[Pipeline] echo
13:52:56 Newly generated image, raspamdocker/osfy
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage

```

Figure 9: Console output

```

20 stage('test') {
21     steps {
22         script {
23             // https://hub.docker.com/r/tutum/hello-world/
24             def container = image.run("-g 80")
25             def contport = container.port(80)
26             println image.id + " container is running at host port, " + contport
27         }
28     }
29 }

```

Figure 10: Start a new container

while introducing this public image hosted at Docker Hub – the image runs Apache at Port 80.

If you were building the image locally in your laptop or desktop, then a new container can be started using the image. Get to know the port using the `docker port` command, and test it using a Web browser by pointing to the appropriate port on the local host. It is port 80 in the running container but different in the local host. The challenge lies in how to simulate this flow in a CI environment.

Starting a Docker container

Let's start a container that's based on the newly built image and use the `curl` command with a few options (which will be explained), as shown in Figure 10. Start a new container to simulate (programmatically using pipeline-as-code) what was possible in a Web browser.

Line 33 is another example to show the usage of a function from the Docker variable – this helps to start a new container at Port 80 and its output is stored in a variable container. To know the port mapping on the host, in Line 34, run the pipeline equivalent of the Docker port command.

To reinforce this, let us look at Figure 11, which is an extract of the build console output generated from executing Lines 33 to 35 as part of the full run of the build job.

Note: On every start of a new container (Docker runs using the CLI or by running a new job of the project in Jenkins), using the 'Hello World' image, port mapping in the local host will be different.

Querying Apache's port

From Figure 11, we know the port mapping on the local host (here it is 32788, but could be different in your setup) for this run of the build job. Now, `curl` can be used to hit this port in the local host, i.e., the build agent in the context of the build job being executed—we will be interested to know the HTTP response (code) as confirmation of having reached the running container successfully.

```

[Pipeline] stage
[Pipeline] { (test)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
13:52:56 [tutum-hello-world-anyagent] Running shell script
13:52:56 + docker run -d -p 80 raspamdocker/osfy
[Pipeline] dockerfingerprintfrom
[Pipeline] sh
13:53:00 [tutum-hello-world-anyagent] Running shell script
13:53:00 + docker port 0d6abc738f4b10eac5ecbaa7d992530783c7d25eb1820fb37709aad280ff 80
[Pipeline] echo
13:53:00 raspamdocker/osfy container is running at host port, 0.0.0.0:32788

```

Figure 11: Query the port mapping on the host

```

36     def resp = sh(returnStdout: true,
37                   script: """
38                       set -x
39                       curl -w "%{http_code}" -o /dev/null -s \
40                           http://127.0.0.1:${contport}/
41                       """)
42     resp.trim()

```

Figure 12: Test the running container is a continuation of the block of code in the test stage